LLM ENGINEERING

Week 03

Multi-Head Attention & Positional Encoding

Bethgelab · Thursday, 29 October 2026

University of Tübingen · Winter term 2026/27



Announcements

Next week, 5 November

- Lecture in person again
- Read Bonus 2 before the tutorial

Quiz 1 on ILIAS

- Weeks 1, 2 and Bonus 1
- Due Friday 30 October, end of day

Second exercise: this lecture

- Due Wednesday 4 November, 22:00
- You get an email when it arrives

Week 2 built one head and one block

- One attention head, width `D`
- One pre-norm block, final RMSNorm, tied head
- Position added once, at the input
- Each query: one softmax row



How does attention read several things at once, and know where they are?

- Heads: several reads per query
- Shared key and value heads make the cache smaller
- One mask for padded, packed batches
- Position inside the score: rotary embeddings
- Exercise: train six variants and compare them

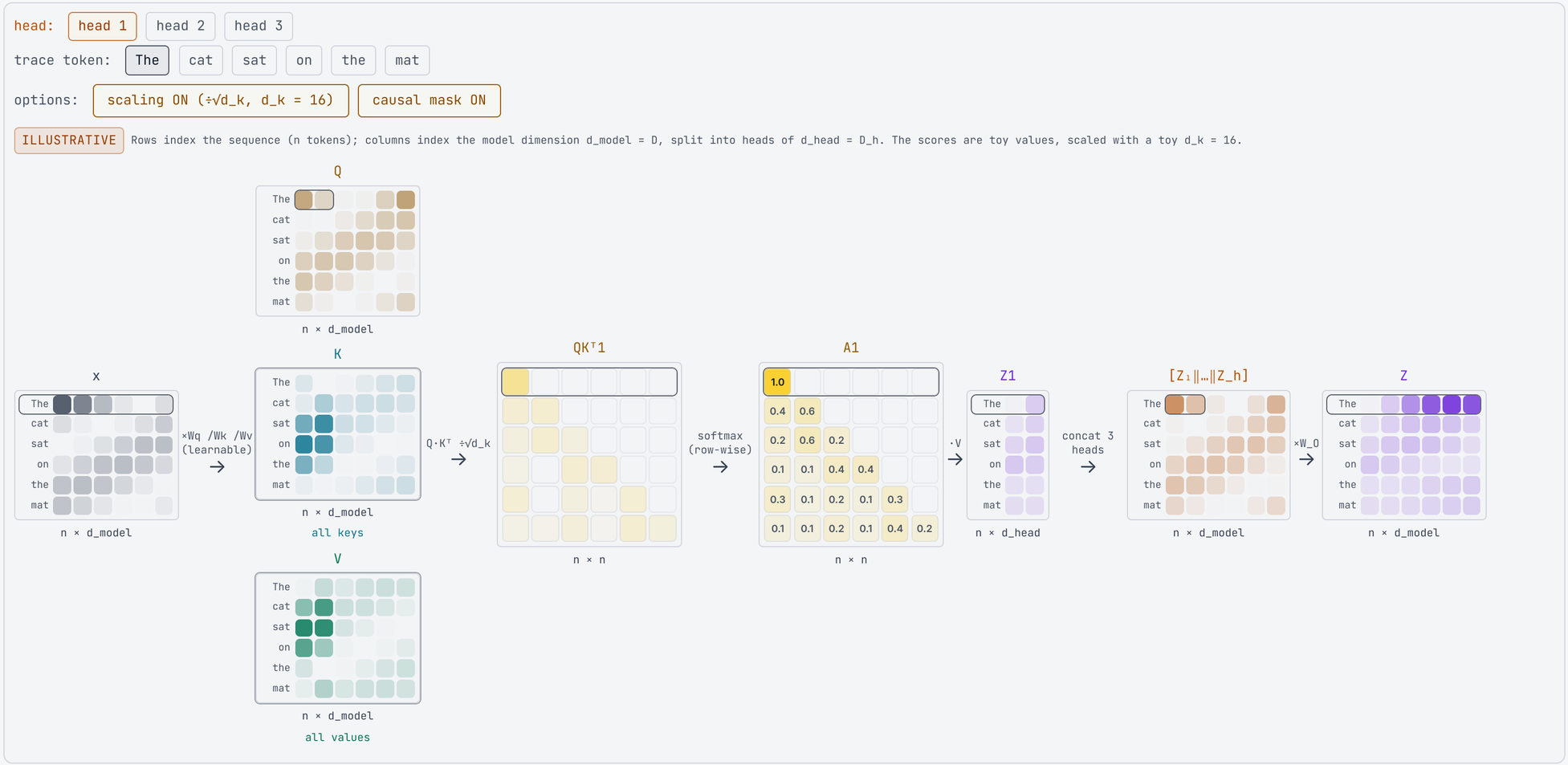


How can attention read two places at once without blending them?



Several heads give several softmax rows

- Each head has its own projections and its own scores
- One softmax row per query, per head
- Concatenate the head outputs, then apply `W_O`
- No fixed job per head



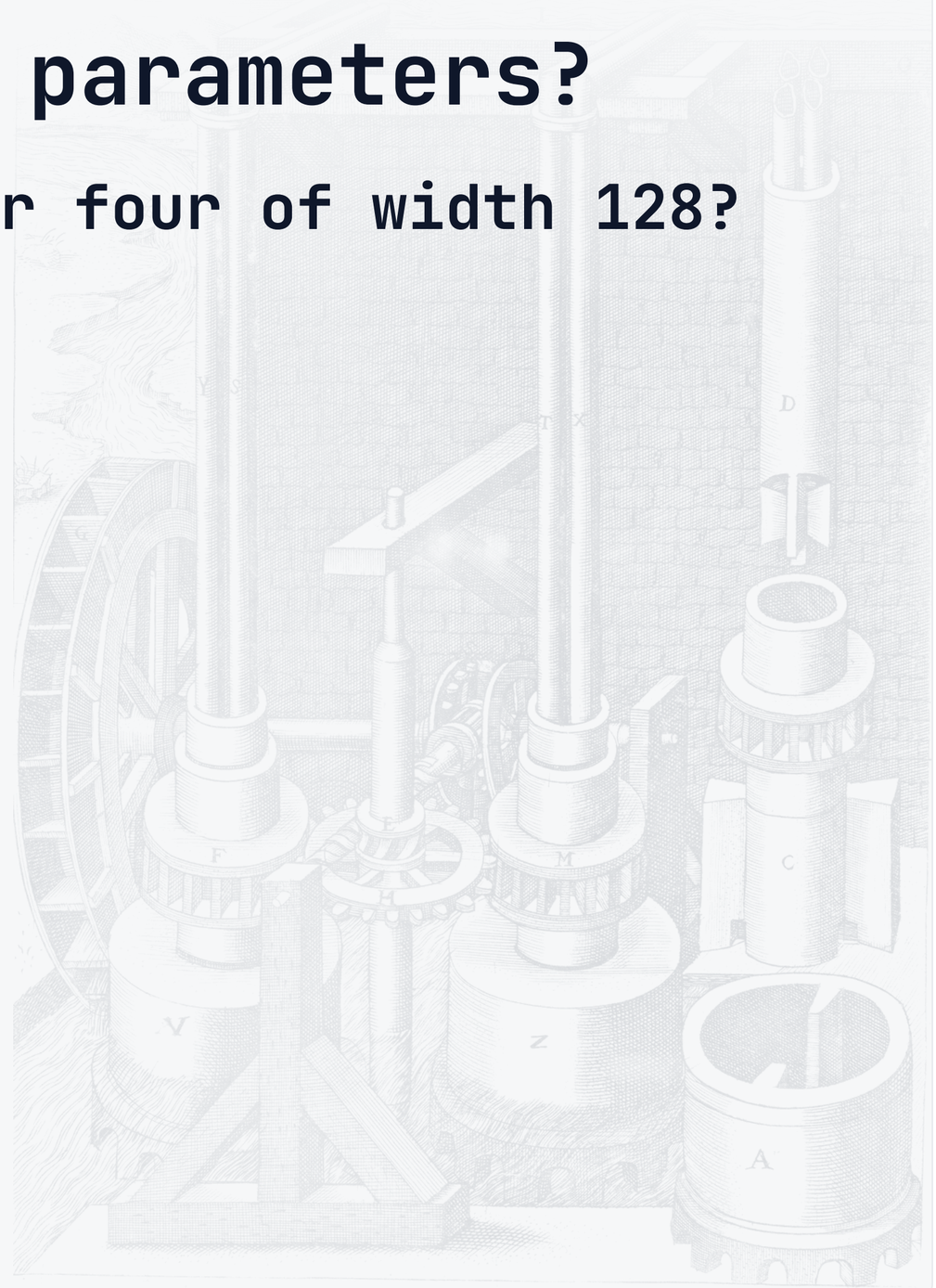
The Idea: Attend in Parallel Subspaces

```
Q, K, V     [B,S,D]
reshape     [B,S,H,D_h]
transpose   [B,H,S,D_h]
scores      [B,H,S,S]
context     [B,H,S,D_h]
transpose   [B,S,H,D_h]
concat      [B,S,D]  → W_O
```

- **Multi-head attention** (MHA): head width `D_h = D/H` (Vaswani et al., 2017)
- Each head gets a slice of the learned projections
- Scale by `√D_h`, not `√D`

Which has more parameters?

One head of width 512, or four of width 128?



Four narrow heads cost the same as one wide head

Quantity	one width-D head	H width-D_h heads
projection parameters	$4D^2$	$4D^2$
dense attention work	$O(S^2D)$	$O(S^2D)$
channels per head	D	D/H

- Same parameters and work, but each head is narrower

One more token: what is new?

Which scores, keys and values must be computed?

ILLUSTRATIVE | Prefill | Decode 1 | Decode 2

keys (columns)

queries (rows)	k_1 The	k_2 cat	k_3 sat	k_4 on
q_1 The	1.00			
q_2 cat	0.44	0.56		
q_3 sat	0.15	0.53	0.32	
q_4 on	0.11	0.14	0.47	0.29

keys



values

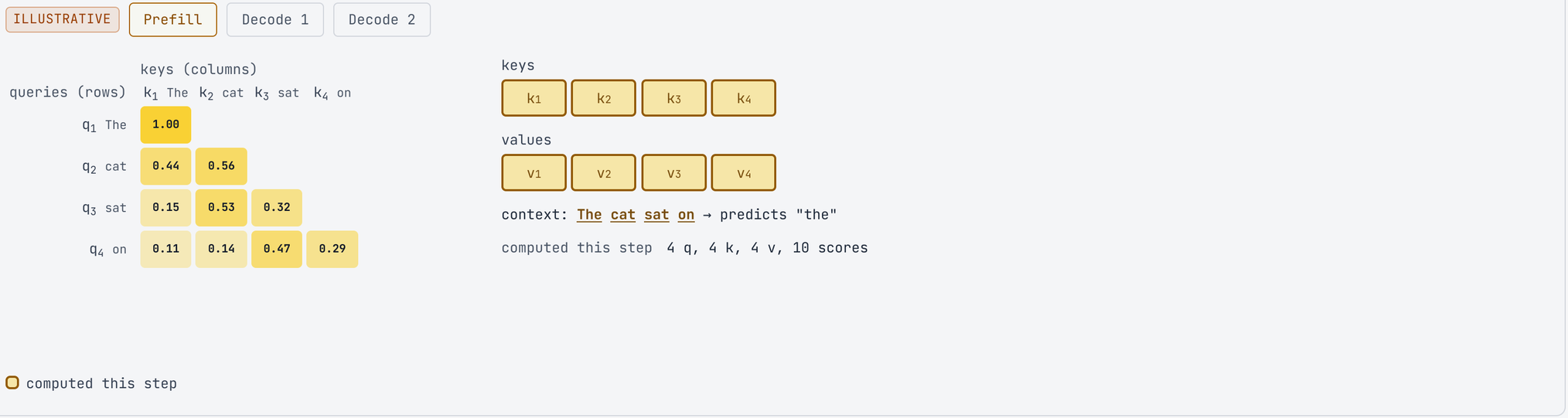


context: The cat sat on → predicts "the"

computed this step 4 q, 4 k, 4 v, 10 scores

computed this step

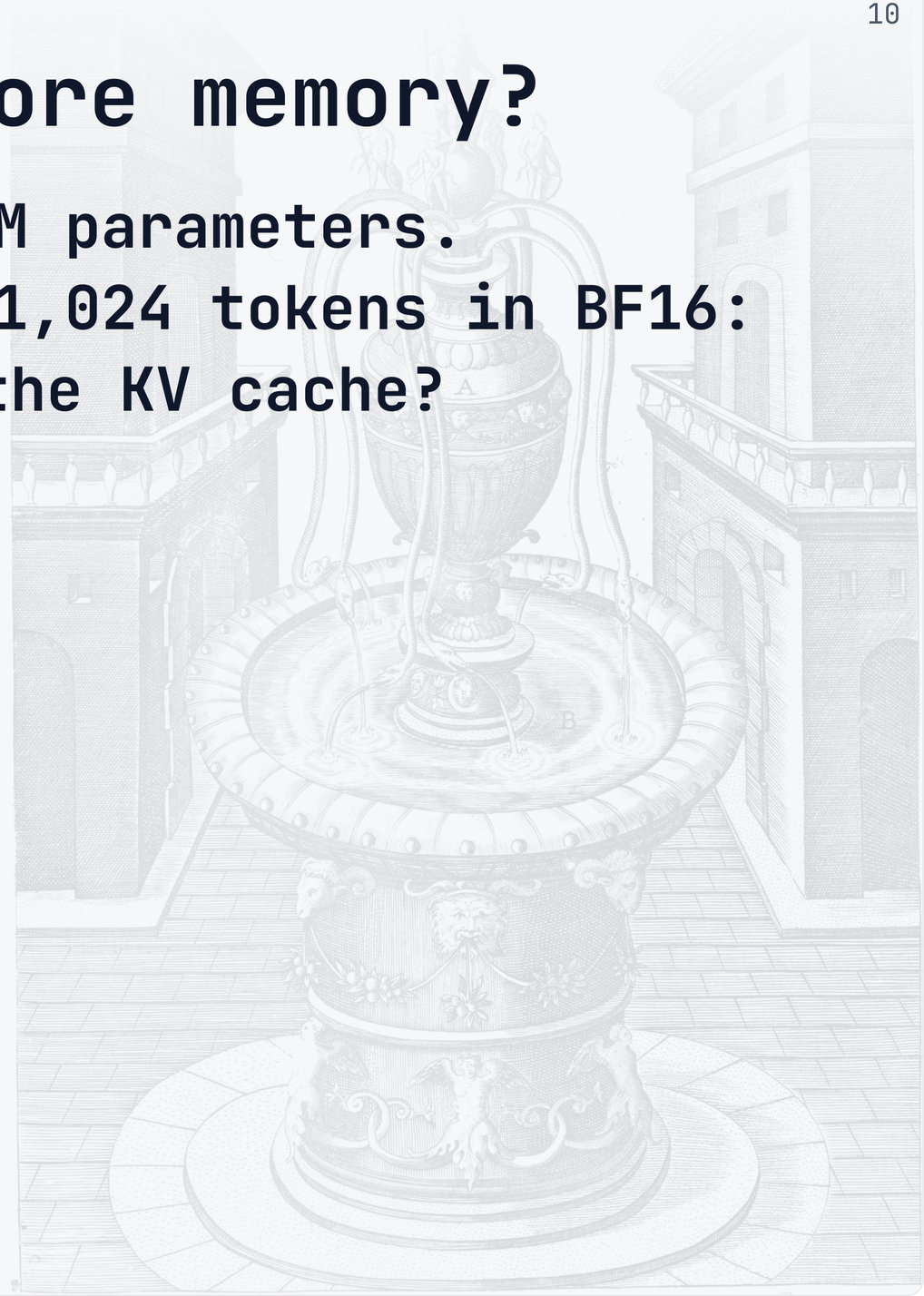
Decode adds one row and reads the cache



- Prefill computes all rows of the prompt at once
- **Decode** adds one new query row per token
- **Key-value (KV) cache**: keys and values of past tokens
- Earlier rows are never stored and never recomputed

Which needs more memory?

Course model, 25M parameters.
Serving 32 sequences of 1,024 tokens in BF16:
the weights or the KV cache?



Here the KV cache is ten times the weights

- KV cache: 512 MiB
- Weights: about 48 MiB
- Here the cache is about 10× larger

Every KV head costs cache memory

KV cache

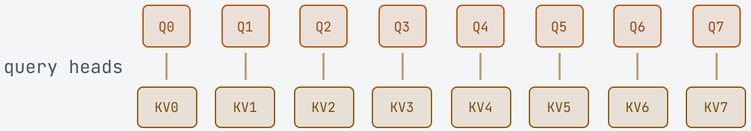
$$\text{KV bytes} = 2BSLH_{kv}D_h \text{ bytes/element}$$

- Cached: K, V of every past token
- Grows with batch, context, depth
- One factor to cut: `H_kv`

Query heads can share key and value heads

- MHA: `n_kv_heads = n_heads`
- **Grouped-query attention** (GQA): `n_kv_heads` between 1 and `n_heads`, dividing `n_heads` (Ainslie et al., 2023)
- **Multi-query attention** (MQA): `n_kv_heads = 1` (Shazeer, 2019)
- Each KV head serves `g = H/H_kv` query heads

ILLUSTRATIVE variant: MHA | GQA | MQA



query heads	KV heads	KV cache vs MHA
8	8	8/8 (full)

What GQA Saves

- `D=512`, `H=4`, `D_h=128`, `L=8`, `S=1024`, BF16

Variant	projection params / layer	cache `B=1`	cache `B=32`
MHA (`H_kv=4`)	1,048,576	16 MiB	512 MiB
GQA (`H_kv=2`)	786,432	8 MiB	256 MiB
MQA (`H_kv=1`)	655,360	4 MiB	128 MiB

- QK and AV work unchanged
- Speed depends on the implementation
- Quality: compare loss curves (Ainslie et al., 2023)

One rule decides every allowed pair

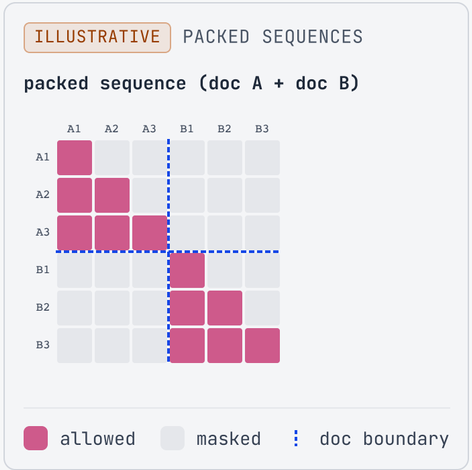
$$\begin{aligned}\text{allowed}[b, q, k] = \text{valid_query}[b, q] \wedge (k \leq q) \\ \wedge \text{valid_key}[b, k] \wedge \text{same_document}[b, q, k]\end{aligned}$$

Which keys may B2 read?

Packed row: A1 A2 A3 B1 B2 B3

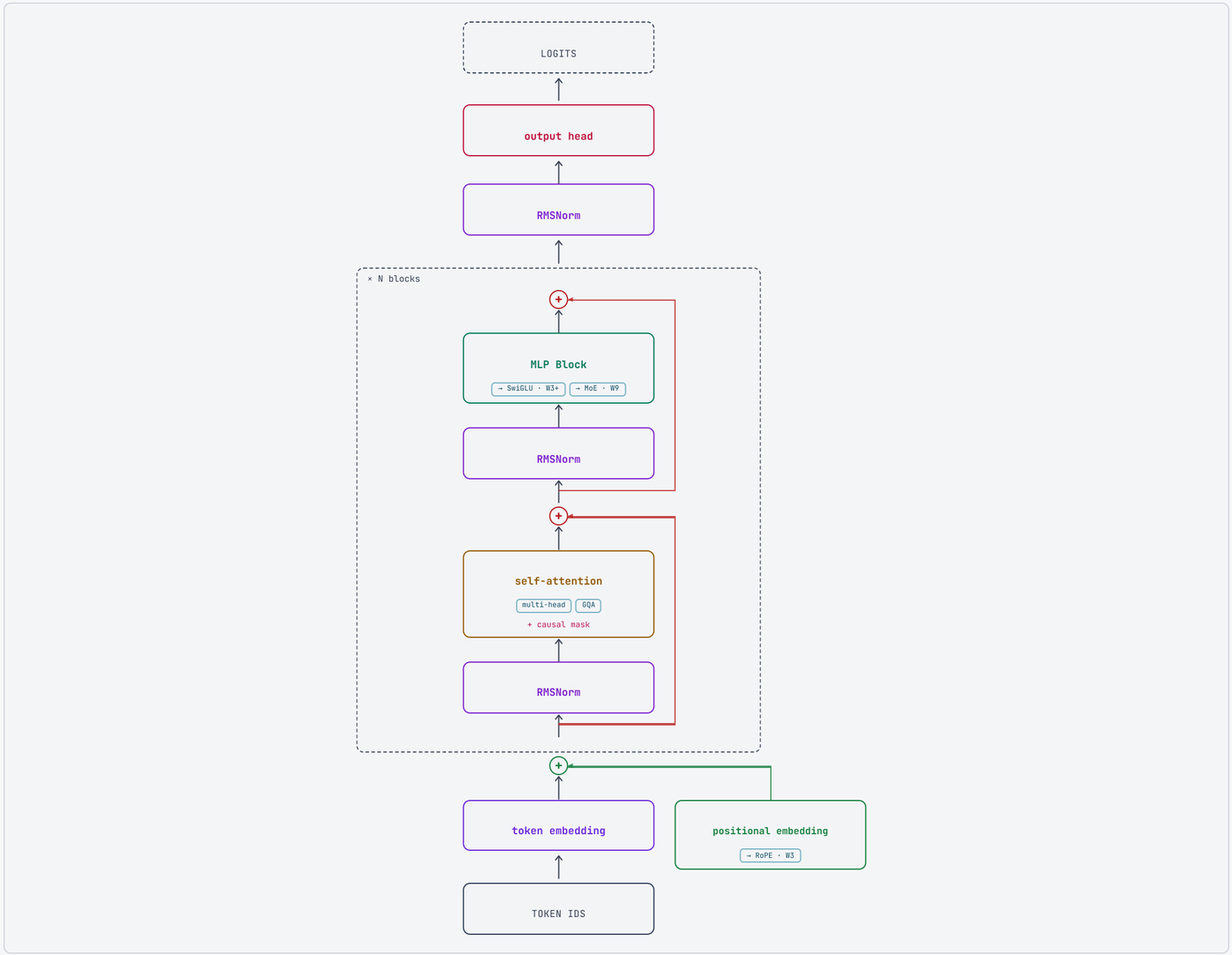
Packed documents never see each other

- B2 reads B1 and B2 only
- Block-diagonal: one block per document
- Restart positions per document, skip targets across boundaries
- Rebuild the mask for every batch



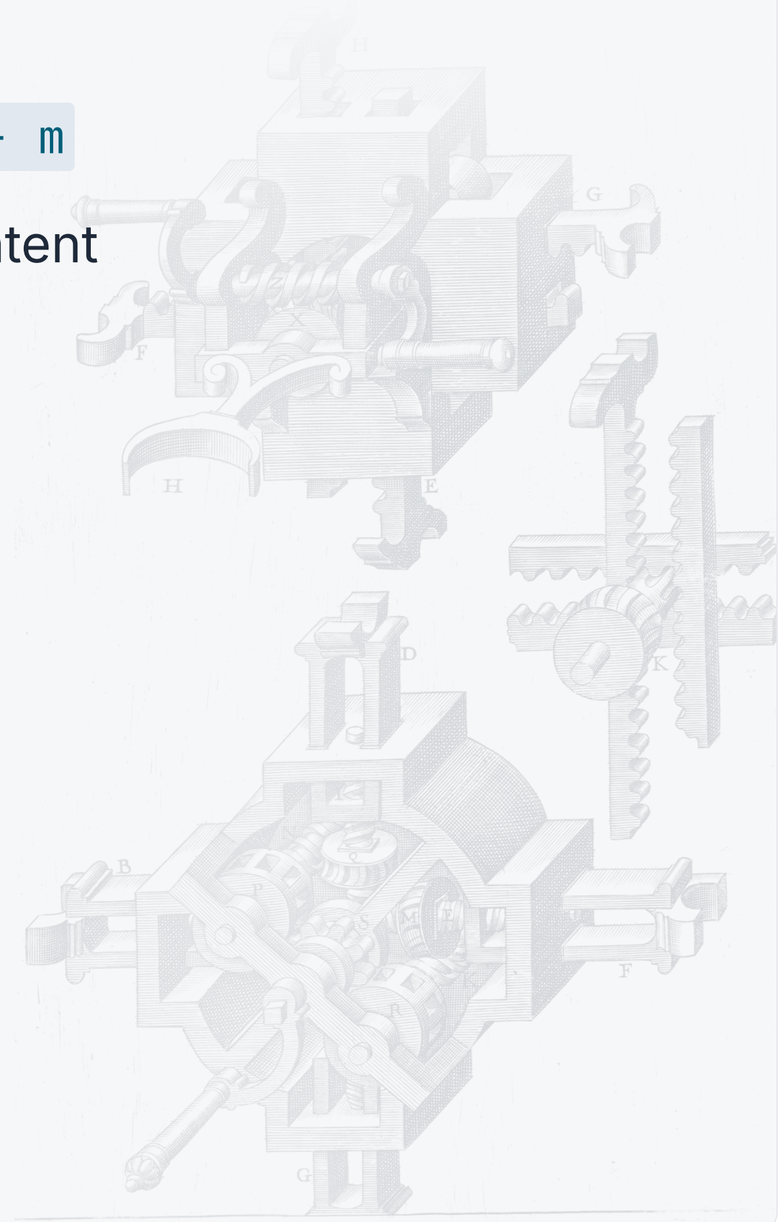
Repeat the block, then normalize once

- The same block repeats `L` times, keeping `[B,S,D]`
- Final RMSNorm, then the tied head
- Later layers build on earlier ones



The score should depend on the offset

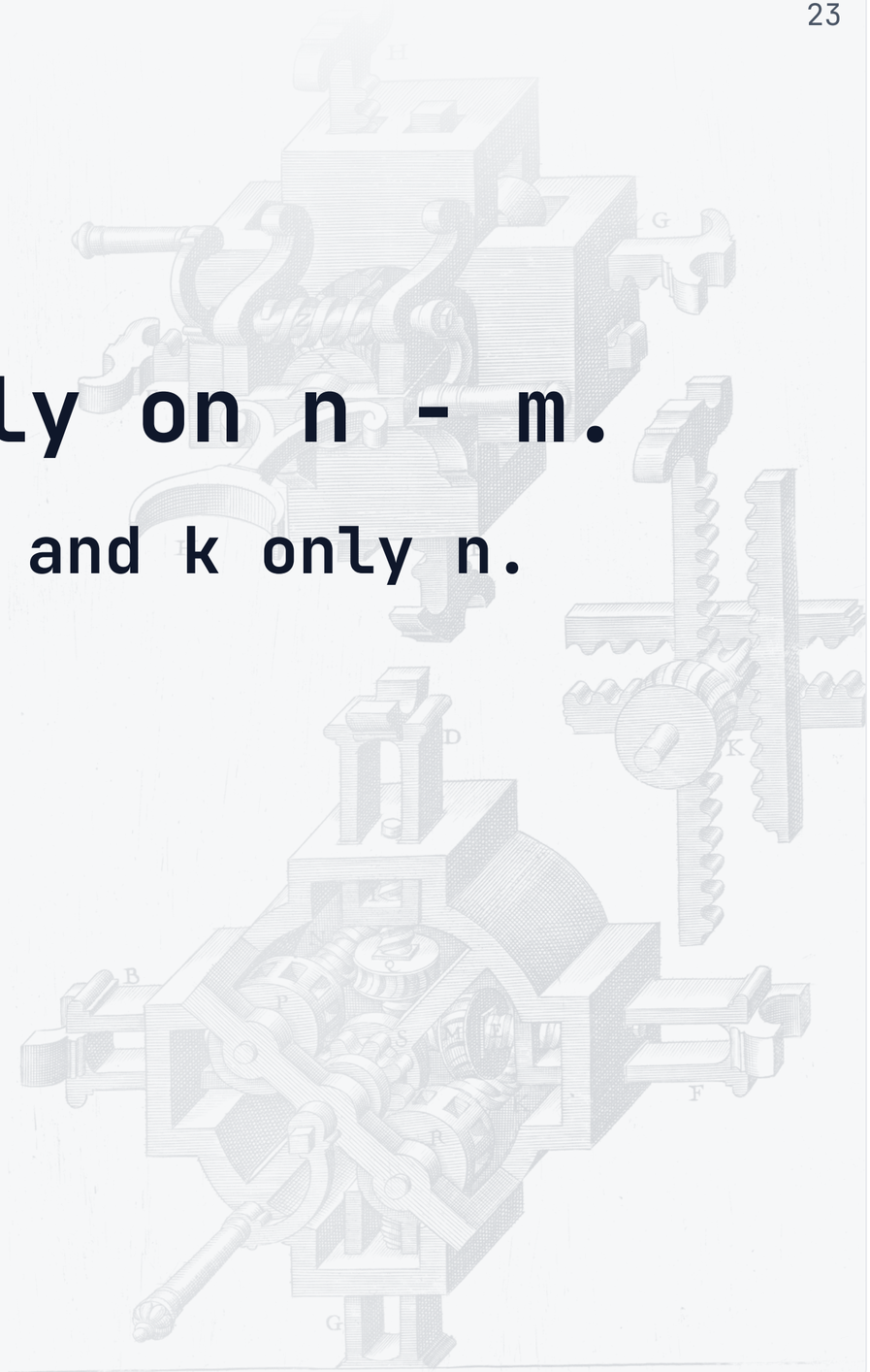
- Score compares query `m` with key `n`
- Language structure mostly depends on the offset `n - m`
- Sinusoidal position is added once and mixed with content
- Nothing forces the score onto `n - m`



Make q·k depend only on n - m.

Rule: q may use only m, and k only n.

- Work in 2D: q and k as arrows
- What sets `q·k`: lengths, angle, both?
- What turns an arrow, keeps its length?



Rotating q and k leaves only the offset

column-vector rotation convention

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

relative-rotation derivation

$$R(m\omega)^\top R(n\omega) = R(-m\omega)R(n\omega) = R((n-m)\omega)$$

relative-score identity

$$(R_m q)^\top (R_n k) = q^\top R_{n-m} k, \quad R_m = R(m\omega)$$

- Rotate Q and K, never V
- Full head: rotate all `D_h` coordinates, in pairs
- The identity holds for one score term (Su et al., 2024)

Each pair turns at its own frequency

pair frequency

$$\omega_j = b^{-2j/d_r}$$

pair rotation

$$R(m\omega_j) = \begin{pmatrix} \cos(m\omega_j) & -\sin(m\omega_j) \\ \sin(m\omega_j) & \cos(m\omega_j) \end{pmatrix}$$

- Base `b = 10000` in the course
- Pair 0 fastest, the last pair slowest
- Fixed angles, no learned parameters

Fast pairs turn quickly, slow pairs barely move

- Pair 0 turns one radian per position
- Shifting both positions keeps the angle between the two vectors

ILLUSTRATIVE position: 4 0 – 31

rotation per 2D pair, from fast (pair 0, left) to slow (pair 3, right):



pair 0: 229.2° | pair 1: 22.9° | pair 2: 2.3° | pair 3: 0.2°

$\text{angle}_j(m) = m \cdot 10000^{-2j/d_r}$, $d_r = 8$ (illustrative; $j = 0 \ldots 3$)

shift demo: both positions shifted by +5 (pair 0)



enclosed angle (arc): 171.9° in both cases. Angle = $(n - m) \times \text{base}^{-2 \cdot 0/8} = (7 - 4) \times 1 = 3$ rad, which depends only on the offset.

No position signal at all.

Can a causal decoder tell where a token is?

A causal mask alone carries some position

- Partly: query `t` sees `t + 1` keys
- Without the mask, attention is order-blind

ILLUSTRATIVE Pattern: **RoPE** NoPE

SYNTHETIC ROPE MATRIX: IDENTICAL CONTENT, COSINE SCORES

	The	cat	sat	on	the	mat
The	1.00					
cat	0.42	0.58				
sat	0.18	0.35	0.48			
on	0.10	0.16	0.31	0.43		
the	0.12	0.09	0.14	0.27	0.38	
mat	0.19	0.10	0.08	0.11	0.22	0.31

Probes find position in NoPE models

- Probes recover position in the models studied (Haviv et al., 2022)
- **NoPE** generalizes best to longer sequences: ten tasks, 100M-parameter models (Kazemnejad et al., 2023)
- Your run: one training-loss comparison

Position is a configuration choice

- `rope`: the base from Week 3 on
- `sinusoidal`: the Week 2 baseline
- `nope`: ablation and design option

Position and locality are separate choices



Six Takeaways

1. Heads: split, attend, concatenate, W_O; same $4D^2$
2. Fewer KV heads: cache shrinks by H/H_{kv}
3. One composed mask; loss skips document boundaries
4. L pre-norm blocks, one final RMSNorm
5. RoPE: rotate Q, K; score sees offset
6. NoPE: mask gives some order; locality separate



To-dos until next week

Muddiest point

Do it right now! (2 min)

Before the tutorial

- Write down one question to discuss
- Think about the reflection questions

Self-study

- Read the reading slides and notes
- Read Bonus 2 before the Week 4 tutorial

Deadlines

- Quiz 1: Friday 30 October
- Second exercise: Wednesday 4 November, 22:00

Reflection questions

1. At fixed model width, what changes when the number of attention heads increases?
2. Long-context serving is KV-cache bound, and a teammate proposes moving from four KV heads to one. Using this week's arithmetic, what exactly does that change and what does it leave unchanged? Name what the arithmetic cannot tell you, and say what you would run to bound it before shipping.
3. How should causal, padding, document-boundary, and loss masks compose?
4. A packed-document run trains for hours, then returns NaN loss on a batch whose last document is unusually short. Which two mask terms are your candidates, where would you place an assertion that separates them, and why did hours of healthy training not rule this out earlier?
5. Why does rotary position embedding encode relative offsets even though it rotates queries and keys by absolute positions?
6. You must choose `pos_encoding` for a long-context run this week. Your evidence is a controlled study over ten tasks with 100M-parameter decoders and one frontier model's reported layer configuration, and neither matches your workload. What does each source actually license you to conclude, which would you weight more heavily and why, and what would you measure on your own run to find out you were wrong?